

58. Une partition qui avance seule

`jouer()` la lance, et le jeu fait autre chose pendant ce temps.

1. Ce qu'il faut comprendre

C'est toute la différence avec la mélodie à la main : là-bas le programme menait chaque note ; ici `jouer()` lance la partition et **elle avance toute seule**.

Un pas par élément. `"=="` tient la note d'avant, `"--"` fait un silence. Le troisième argument de `jouer()` est le nombre d'images par pas — le tempo.

Dans la page, cette partition s'écrit **à la souris**, sur une portée. L'atelier produit exactement ces lignes.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Air>` → [53.1. Air — écrire une musique](#)
- `#include <jouer>` → [53.2. La fonction jouer\(\) — jouer un air tout seul](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **859 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      71 o ← la console, à l'allumage — la mise en route de la console
main()         70 o ← démarrage — ta fonction
VBlank         11 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AvancerAirs    308 o ← VBlank — fait avancer la musique à chaque image
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         96 o ← CopierTuiles — 6 lettres sur 43
Notes          120 o ← AvancerAirs — la table des notes de musique
"CA JOUE"      7 o ← texte() — un texte gravé
FANFARE_0      20 o ← jouer() — une table gravée
```

Lettres gravées : ACEJOU

Pas gravé (jamais appelé) : AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert

Fonctions du langage appelées :

```
jouer() × 1    52 o sur place, dans main() + FANFARE_0 (20 o)
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : joue une petite fanfare toute seule, une fois.
//
// Un Air est une liste de PAS de musique :
//  D04 12 : la note do (octave 4), volume 12
//  ==    : la note d'avant continue (une note plus longue)
//  --    : silence
// jouer(voix, AIR, vitesse) : joue l'air sur la voix, un pas toutes les « vitesse » images.
// La console le joue TOUTE SEULE : le programme n'a plus rien à faire.

#include <Air>      // un air de musique, note par note
#include <jouer>    // joue un air tout seul
#include <texte>    // écrit un texte à l'écran
```

2. UNE MÉLODIE — FANFARE

```
Air FANFARE = {
    "D04 12", "==", "M14 12", "==", "S0L4 12", "==", "D05 12", "==", "==", "--",
};
```

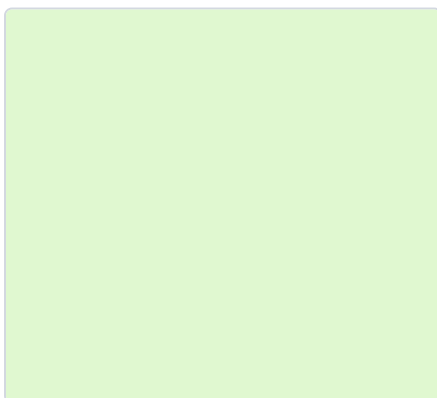
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    jouer(1, FANFARE, 8);    // Voix 1, un pas toutes les 8 images.
    texte(4, 8, "CA JOUE");

    while (true) {          // Répète pour toujours :
        image();             // attend l'image suivante (la musique continue seule).
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une fanfare qui monte, sans que le programme s'en occupe.

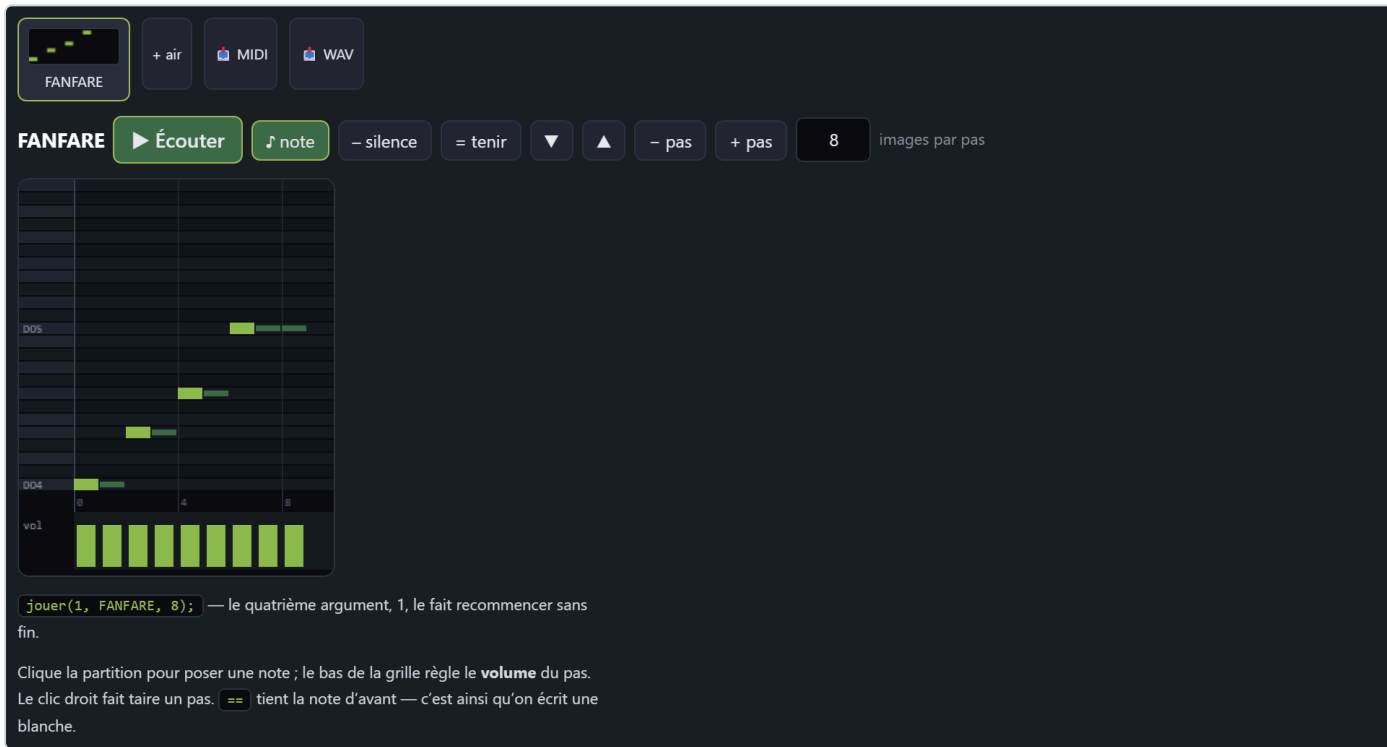
5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le mot est écrit
- ✓ la voix 1 joue
- ✓ elle change de note toute seule (329, 392, 524 Hz)
- ✓ et la mélodie monte
- ✓ sans une ligne dans la boucle du programme

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.



The screenshot shows a music notation software interface for Game Boy. At the top, there are buttons for '+ air', 'MIDI', and 'WAV'. Below these, there's a 'FANFARE' label and a green 'Écouter' button. To the right of 'Écouter' are buttons for 'note', 'silence', '= tenir', and a set of arrow buttons for pitch bending: '▼', '▲', '– pas', and '+ pas'. Further right is a '8' button and the text 'images par pas'. The main area is a piano roll with a grid. The vertical axis is labeled 'vol' (volume) and has markers for '0', '4', and '8'. The horizontal axis represents time. There are several green rectangular notes on the grid. Below the piano roll, there's a text box with the code `jouer(1, FANFARE, 8);` and an explanation: '— le quatrième argument, 1, le fait recommencer sans fin.' Below this, there's more text: 'Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas. Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.'

Ensuite : **58.1. La fonction `airFin()` — l'air est-il fini ?** — chapitre 8 : Le son. · gameboy3 — écrire une cartouche Game Boy en C++